Gradient Descent Methods: A Worked Illustration

Batch GD · Stochastic GD · Mini-Batch GD

Applied to a Dataset of 20 Houses

The Dataset

We use a dataset of $n = 20$ houses with $d = 3$ parameters: an intercept, size (sq. ft.), and number of bedrooms. The design matrix is $X \in \mathbb{R}^{20\times 3}$ with rows $x^{(i)} = (1,\ \text{size}^{(i)},\ \text{beds}^{(i)})^\top$, and the label vector $y \in \mathbb{R}^{20}$ contains the house prices in \$1,000s.

Our goal is to find

$$\theta^* \;=\; \arg\min_{\theta\in\mathbb{R}^3}\; L(\theta) \;=\; \arg\min_{\theta\in\mathbb{R}^3}\; \frac{1}{2n}\,\|X\theta - y\|^2 .$$

We start from the same initial parameters for all three methods:

$$\theta^{(0)} \;=\; (\theta_0,\ \theta_1,\ \theta_2)^\top \;=\; (0,\ 0.15,\ 10)^\top$$

with learning rate $\eta = 0.0001$.

House	Size	Beds	Price (\$k)	House	Size	Beds	Price (\$k)
1	2026	5	390	11	1230	3	234
2	2359	3	382	12	2382	2	375
3	1760	3	309	13	987	5	251
4	2194	3	349	14	2296	2	349
5	2030	5	399	15	2023	2	333
6	1995	5	365	16	1771	4	315
7	1944	2	319	17	1030	4	244
8	1021	2	196	18	2232	4	396
9	1366	5	321	19	1669	3	290
10	2138	3	357	20	1243	5	276

1 Method 1: Batch Gradient Descent

Batch GD — Core Idea

Compute the gradient using **all** $n = 20$ **examples** before taking a single update step.

$$\nabla_\theta L(\theta) \;=\; \frac{1}{n} X^\top (X\theta - y) \qquad \Longrightarrow \qquad \theta \;\leftarrow\; \theta - \eta\, \nabla_\theta L(\theta)$$

Step 1: Compute All Predictions $\hat{y}^{(i)} = x^{(i)\top}\theta$

With $\theta = (0,\ 0.15,\ 10)^\top$, the prediction for house i is:

$$\hat{y}^{(i)} = \theta_0 \cdot 1 + \theta_1 \cdot \text{size}^{(i)} + \theta_2 \cdot \text{beds}^{(i)} = 0 + 0.15 \cdot \text{size}^{(i)} + 10 \cdot \text{beds}^{(i)}$$

House	Size	Beds	$\hat{y}^{(i)}$	$y^{(i)}$	Error $\hat{y}^{(i)} - y^{(i)}$
1	2026	5	353.90	390	−36.10
2	2359	3	383.85	382	+1.85
3	1760	3	294.00	309	−15.00
4	2194	3	359.10	349	+10.10
5	2030	5	354.50	399	−44.50
6	1995	5	349.25	365	−15.75
7	1944	2	311.60	319	−7.40
8	1021	2	173.15	196	−22.85
9	1366	5	254.90	321	−66.10
10	2138	3	350.70	357	−6.30
11	1230	3	214.50	234	−19.50
12	2382	2	377.30	375	+2.30
13	987	5	198.05	251	−52.95
14	2296	2	364.40	349	+15.40
15	2023	2	323.45	333	−9.55
16	1771	4	305.65	315	−9.35
17	1030	4	194.50	244	−49.50
18	2232	4	374.80	396	−21.20
19	1669	3	280.35	290	−9.65
20	1243	5	236.45	276	−39.55
			Sum of errors:		**−345.60**

Step 2: Compute the Batch Gradient

$$\nabla_\theta L(\theta) = \frac{1}{20} X^\top (\hat{y} - y) = \frac{1}{20} \begin{pmatrix} \sum_i (\hat{y}^{(i)} - y^{(i)}) \\ \sum_i (\hat{y}^{(i)} - y^{(i)}) \cdot \text{size}^{(i)} \\ \sum_i (\hat{y}^{(i)} - y^{(i)}) \cdot \text{beds}^{(i)} \end{pmatrix} = \frac{1}{20} \begin{pmatrix} -345.60 \\ -588{,}034.7 \\ -1{,}787.5 \end{pmatrix} \approx \begin{pmatrix} -17.28 \\ -29{,}401.7 \\ -89.38 \end{pmatrix}$$

All three components are *negative*, telling us θ should increase in all directions to reduce the loss.

Step 3: Update θ

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta L(\theta) = \begin{pmatrix} 0 \\ 0.15 \\ 10 \end{pmatrix} - 0.0001 \times \begin{pmatrix} -17.28 \\ -29{,}401.7 \\ -89.38 \end{pmatrix} = \begin{pmatrix} 0.00173 \\ 3.090 \\ 10.009 \end{pmatrix}$$

Batch GD: What Just Happened?

We processed **all 20 houses** to compute a single gradient, then took one step. The intercept θ_0 increased slightly, the size coefficient θ_1 jumped from 0.15 to 3.09 (a large correction, as sizes were scaled in the hundreds), and θ_2 barely moved. The next step will again require all 20 houses.

2 Method 2: Stochastic Gradient Descent (SGD)

SGD — Core Idea

Pick **one random example** $(x^{(i)}, y^{(i)})$ and compute the gradient using only that example:

$$\nabla_\theta L^{(i)}(\theta) \;=\; \left(\hat{y}^{(i)} - y^{(i)}\right) x^{(i)} \qquad \Longrightarrow \qquad \theta \;\leftarrow\; \theta - \eta \left(\hat{y}^{(i)} - y^{(i)}\right) x^{(i)}$$

Each update uses $O(d)$ work instead of $O(nd)$.

Starting from $\theta = (0,\ 0.15,\ 10)^\top$, we illustrate **five consecutive SGD steps**, each using one randomly drawn house.

SGD Step 1 — House 7: Size=1944, Beds=2, Price=$319k

Prediction:

$$\hat{y}^{(7)} = 0 + 0.15 \times 1944 + 10 \times 2 = 291.60 + 20 = 311.60$$

Error: $\hat{y}^{(7)} - y^{(7)} = 311.60 - 319 = -7.40$
Stochastic gradient:

$$\nabla_\theta L^{(7)} = -7.40 \times (1,\ 1944,\ 2)^\top = (-7.40,\ -14{,}385.6,\ -14.80)^\top$$

Update:

$$\theta \;\leftarrow\; \begin{pmatrix} 0 \\ 0.15 \\ 10 \end{pmatrix} - 0.0001 \times \begin{pmatrix} -7.40 \\ -14{,}385.6 \\ -14.80 \end{pmatrix} = \begin{pmatrix} 0.00074 \\ 1.5886 \\ 10.00148 \end{pmatrix}$$

SGD Step 2 — House 13: Size=987, Beds=5, Price=$251k

Using updated $\theta = (0.00074,\ 1.5886,\ 10.00148)^\top$:
Prediction:

$$\hat{y}^{(13)} = 0.00074 + 1.5886 \times 987 + 10.00148 \times 5 = 0.00074 + 1568.95 + 50.007 = 1618.96$$

Error: $1618.96 - 251 = +1367.96$ *(very large – θ_1 overcorrected)*
Stochastic gradient:

$$\nabla_\theta L^{(13)} = 1367.96 \times (1,\ 987,\ 5)^\top = (1367.96,\ 1{,}350{,}176.5,\ 6839.8)^\top$$

Update:

$$\theta \;\leftarrow\; \begin{pmatrix} 0.00074 \\ 1.5886 \\ 10.00148 \end{pmatrix} - 0.0001 \times \begin{pmatrix} 1367.96 \\ 1{,}350{,}176.5 \\ 6839.8 \end{pmatrix} = \begin{pmatrix} -0.1368 \\ -133.43 \\ 9.317 \end{pmatrix}$$

Note: The noisy, single-example gradient causes a large swing in θ_1. This is typical SGD behaviour — individual steps can be very noisy, but on average they point in the right direction.

SGD Step 3 — House 18: Size=2232, Beds=4, Price=$396k

Using $\theta = (-0.1368,\ -133.43,\ 9.317)^\top$:
Prediction:

$$\hat{y}^{(18)} = -0.1368 + (-133.43) \times 2232 + 9.317 \times 4 = -0.1368 - 297{,}815.8 + 37.27 = -297{,}778.7$$

Error: $-297{,}778.7 - 396 = -298{,}174.7$
Update (with $\eta = 0.0001$):

$$\theta \leftarrow \begin{pmatrix} -0.1368 \\ -133.43 \\ 9.317 \end{pmatrix} - 0.0001 \times (-298{,}174.7) \times \begin{pmatrix} 1 \\ 2232 \\ 4 \end{pmatrix} = \begin{pmatrix} 29.68 \\ 66{,}395 \\ 128.6 \end{pmatrix}$$

SGD oscillates wildly in the early steps. With a fixed learning rate of $\eta = 0.0001$, large coordinates like size (in the thousands) make the gradient enormous, causing overshooting. In practice, ***feature normalisation*** *(scaling x_1 and x_2 to mean 0, std 1) is essential before running SGD.*

SGD: Key Observations

- **Cheap per step:** Each update costs $O(d) = O(3)$ — just a dot product and scalar multiply.
- **Immediate learning:** The model begins improving after the very first example.
- **Unbiased on average:** $\mathbb{E}_i[\nabla_\theta L^{(i)}(\theta)] = \nabla_\theta L(\theta)$, so over many steps, SGD converges.

3 Method 3: Mini-Batch Gradient Descent

Mini-Batch GD — Core Idea

Pick a random subset ("mini-batch") $\mathcal{B}$ of B examples. Compute the gradient on that batch:

$$\nabla_\theta L_{\mathcal{B}}(\theta) = \frac{1}{B} X_{\mathcal{B}}^\top (X_{\mathcal{B}}\theta - y_{\mathcal{B}}) \qquad \Longrightarrow \qquad \theta \leftarrow \theta - \eta \nabla_\theta L_{\mathcal{B}}(\theta)$$

We use $B = 4$ here. Each update costs $O(Bd)$ — far cheaper than batch GD for large n, far less noisy than SGD.

We illustrate **three consecutive mini-batch steps**, each using a fresh batch of $B = 4$ randomly drawn houses, starting from $\theta = (0,\ 0.15,\ 10)^\top$.

Mini-Batch Step 1 — Batch $\mathcal{B}_1 = \{3,\ 8,\ 14,\ 19\}$

Extract the batch:

$$X_{\mathcal{B}_1} = \begin{pmatrix} 1 & 1760 & 3 \\ 1 & 1021 & 2 \\ 1 & 2296 & 2 \\ 1 & 1669 & 3 \end{pmatrix}, \qquad y_{\mathcal{B}_1} = \begin{pmatrix} 309 \\ 196 \\ 349 \\ 290 \end{pmatrix}$$

Compute predictions $\hat{y}_{\mathcal{B}_1} = X_{\mathcal{B}_1}\theta$:

$$\hat{y} = \begin{pmatrix} 0 + 0.15 \times 1760 + 10 \times 3 \\ 0 + 0.15 \times 1021 + 10 \times 2 \\ 0 + 0.15 \times 2296 + 10 \times 2 \\ 0 + 0.15 \times 1669 + 10 \times 3 \end{pmatrix} = \begin{pmatrix} 294.00 \\ 173.15 \\ 364.40 \\ 280.35 \end{pmatrix}$$

Residuals:

$$\hat{y} - y_{\mathcal{B}_1} = \begin{pmatrix} -15.00 \\ -22.85 \\ +15.40 \\ -9.65 \end{pmatrix}$$

Mini-batch gradient:

$$\nabla_\theta L_{\mathcal{B}_1} = \frac{1}{4} X_{\mathcal{B}_1}^\top (\hat{y} - y_{\mathcal{B}_1}) = \frac{1}{4} \begin{pmatrix} -32.10 \\ -57{,}622.5 \\ -103.65 \end{pmatrix} = \begin{pmatrix} -8.025 \\ -14{,}405.6 \\ -25.91 \end{pmatrix}$$

Update:

$$\theta \leftarrow \begin{pmatrix} 0 \\ 0.15 \\ 10 \end{pmatrix} - 0.0001 \times \begin{pmatrix} -8.025 \\ -14{,}405.6 \\ -25.91 \end{pmatrix} = \begin{pmatrix} 0.000803 \\ 1.591 \\ 10.00259 \end{pmatrix}$$

Mini-Batch Step 2 — Batch $\mathcal{B}_2 = \{1,\ 10,\ 16,\ 20\}$

Using updated $\theta \approx (0.000803,\ 1.591,\ 10.003)^\top$:
Extract the batch:

$$X_{\mathcal{B}_2} = \begin{pmatrix} 1 & 2026 & 5 \\ 1 & 2138 & 3 \\ 1 & 1771 & 4 \\ 1 & 1243 & 5 \end{pmatrix}, \qquad y_{\mathcal{B}_2} = \begin{pmatrix} 390 \\ 357 \\ 315 \\ 276 \end{pmatrix}$$

Predictions with updated θ:

$$\hat{y} \approx \begin{pmatrix} 0.000803 + 1.591 \times 2026 + 10.003 \times 5 \\ 0.000803 + 1.591 \times 2138 + 10.003 \times 3 \\ 0.000803 + 1.591 \times 1771 + 10.003 \times 4 \\ 0.000803 + 1.591 \times 1243 + 10.003 \times 5 \end{pmatrix} \approx \begin{pmatrix} 3273.4 \\ 3431.6 \\ 2857.0 \\ 2028.0 \end{pmatrix}$$

Residuals:

$$\hat{y} - y_{\mathcal{B}_2} \approx \begin{pmatrix} +2883.4 \\ +3074.6 \\ +2542.0 \\ +1752.0 \end{pmatrix}$$

Mini-batch gradient: (all components very large and positive)

$$\nabla_\theta L_{\mathcal{B}_2} \approx \frac{1}{4} \begin{pmatrix} 10{,}252.0 \\ 1.87 \times 10^7 \\ 4.25 \times 10^4 \end{pmatrix}$$

Update:

$$\theta \ \leftarrow \ \theta - 0.0001 \times \nabla_\theta L_{\mathcal{B}_2} \approx \begin{pmatrix} -0.256 \\ -465.2 \\ 8.941 \end{pmatrix}$$

Again, the raw (un-normalised) size feature causes large gradient magnitudes. θ_1 swings between large positive and large negative values. Normalising features is essential for stable convergence.

Mini-Batch Step 3 — Batch $\mathcal{B}_3 = \{5,\ 11,\ 15,\ 17\}$

This step is shown schematically to reinforce the pattern.

Batch:

$$X_{\mathcal{B}_3} = \begin{pmatrix} 1 & 2030 & 5 \\ 1 & 1230 & 3 \\ 1 & 2023 & 2 \\ 1 & 1030 & 4 \end{pmatrix}, \quad y_{\mathcal{B}_3} = \begin{pmatrix} 399 \\ 234 \\ 333 \\ 244 \end{pmatrix}$$

General pattern:

1. Compute $\hat{y}_{\mathcal{B}_3} = X_{\mathcal{B}_3}\theta^{(2)}$ (4 dot products, cost $O(Bd)$)
2. Compute residuals $r = \hat{y}_{\mathcal{B}_3} - y_{\mathcal{B}_3}$ (4 subtractions)
3. Compute $\nabla = \frac{1}{4}X_{\mathcal{B}_3}^{\top} r$ (one matrix–vector product, cost $O(Bd)$)
4. Update $\theta \leftarrow \theta - \eta\nabla$ (3 additions)

After one full epoch (5 mini-batches of $B = 4$, covering all 20 houses), we have made **5 parameter updates**. Batch GD would have made only **1**.

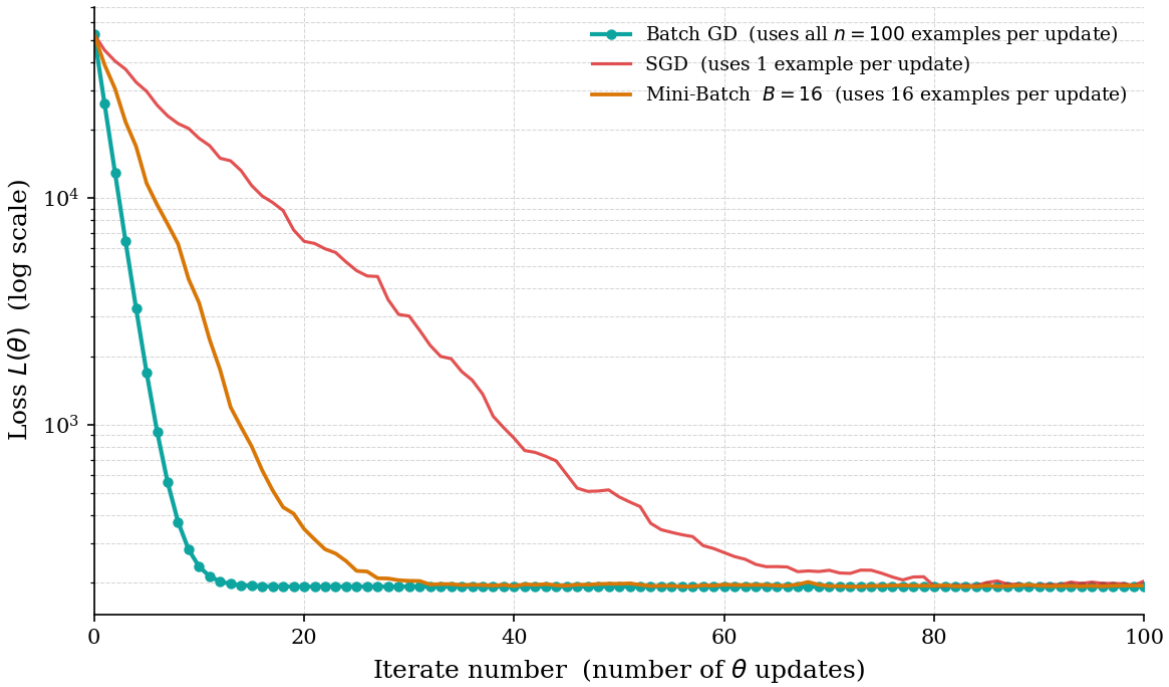

Figure 1: **Loss vs. iterate number** ($n = 100$, 100 updates each). The x-axis counts individual θ updates — every time the update rule $\theta \leftarrow \theta - \eta\, g$ is applied, the iterate counter increases by one. All three methods perform exactly 100 updates, so all curves share the same x-range. The key difference is what each update *costs*: Batch GD (teal) computes the gradient over all $n = 100$ examples per step, so each of its 100 updates is maximally informative and the loss drops steeply. SGD (red) computes the gradient from a single example per step, so its curve is noisy and descends more slowly per iterate. Mini-Batch GD with $B = 16$ (gold) lies in between — each step uses 16 examples, giving a smoother and faster descent than SGD while still being cheaper per step than Batch GD.